LoRA: Low-Rank Adaptation of Large Language Models

Reproducing parameter-efficient fine-tuning on RoBERTa-base + SST-2 Sentiment Classification

Cornell University · CS 4782
Final Project · Spring 2026

Elliott Lin (el729) · Anne Xia (ax53) · Trisha Saini (tfs47) · Samarth Shridhar (shs96) | CS 4782 · Spring 2026

MOTIVATION & BACKGROUND

Why full fine-tuning doesn't scale

Compute
Gradients touch every param

Memory
AdamW stores 2× params

Storage
Full ckpt per task

Speed
Slow loops = few experiments

The LoRA Solution

$$h = W_0 x + (\alpha/r) \cdot BAx$$

B init: zeros · A init: Gaussian · W_0: frozen · α/r: scaling

Our Hypothesis

LoRA on RoBERTa-base reaches comparable SST-2 validation accuracy to full fine-tuning while training only ~0.82% of parameters, using less GPU memory, and training faster.

SST-2 Dataset

Stanford Sentiment Treebank · binary sentiment (positive / negative)
67k train · 872 test · via HuggingFace stanfordnlp/sst2

METHODOLOGY

LoRA Adapter Core Implementation

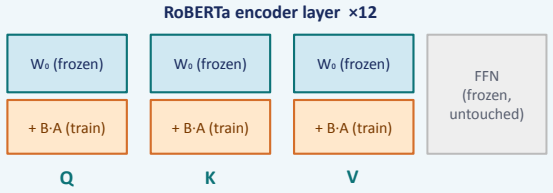
```
class LoRA_Adapter(nn.Module):
  def __init__(self, linear, rank, alpha=None):
    super().__init__()
    self.scaling = (alpha or rank) / rank
    self.pretrained_linear = linear   # frozen at outer level

    # A: (r, in)  Gaussian      B: (out, r)  zeros
    self.lora_A = nn.Parameter(torch.empty(rank, in_features))
    self.lora_B = nn.Parameter(torch.zeros(out_features, rank))
    nn.init.normal_(self.lora_A, std=1/math.sqrt(rank))

  def forward(self, x):
    base = self.pretrained_linear(x)          # Wox
    update = F.linear(F.linear(x, self.lora_A), self.lora_B)
    return base + self.scaling * update       # h
```

METHODOLOGY (CONTINUED)

Patched onto Q, K, V × 12 Encoder Layers



Trainable: 72 tensors (Q,K,V×12) + 4 head tensors ≈ 1.03M / 125M = 0.82%

EXPERIMENTAL SETUP & RESULTS

Hyperparameters & Training Pipeline

Param	Value	Param	Value
Optimizer	AdamW	Batch size	16
Learning rate	5×10^{-4}	Max seq len	512
Weight decay	0.01	Grad clip	1.0
LR schedule	6% warmup →decay	Dropout	0.1 (head)
LoRA α	8	Epochs	10 (paper: 30–60)
Hardware	Colab T4 / A100	LoRA ranks	r ∈ {2,4,6,8}

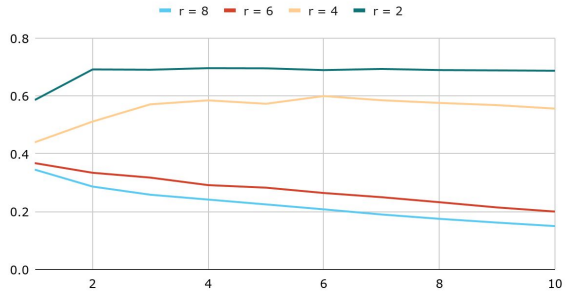
Data splits — Train: 65,349 · Val: 2,000 (carved) · Test: 872

Results: LoRA vs Full Fine-Tuning

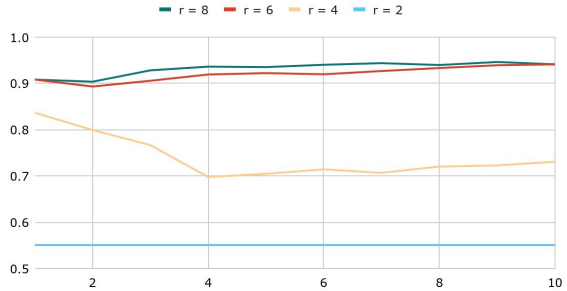
Method	Trainable Params	% of RoBERTa
LoRA r = 2	~0.70M (702,722)	0.56%
LoRA r = 4	~0.81M (813,314)	0.65%
LoRA r = 6	~0.92M (923,906)	0.74%
LoRA r = 8	~1.03M (1,034,498)	0.82%
Full RoBERTa-base	~125M	100%

EXPERIMENTAL SETUP & RESULTS (CONTINUED)

Training loss per epoch

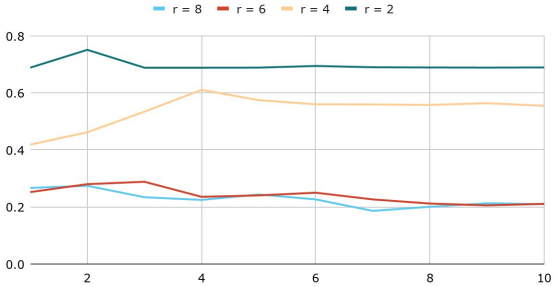


Validation accuracy per epoch



Training and validation accuracy across 10 epochs for r ∈ {2, 4, 6, 8}, α = 8 fixed.

Validation loss per epoch



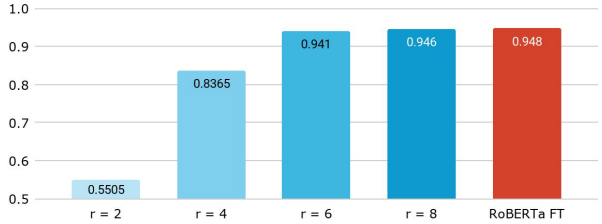
Validation loss mirrors accuracy: r=6 and r=8 converge smoothly and overlap; r=4 oscillates; r=2 stays flat at its initialization.

CONCLUSION

Key Takeaways

1. **LoRA reproduces on lower compute**
2. **Tiny parameter count**
3. **Small rank is forgiving**

Max validation accuracy across epochs



** RoBERTa FT results taken from paper*

LoRA r=8 reaches ~94% val acc and ~91% test acc using only 0.82% of full fine-tuning parameters, closely validating the paper's core claim.

LIMITATIONS

- 10 epochs vs. 30-60 in paper
- Only tested RoBERTa-base and SST-2
- FT baseline taken from paper's longer runs

FUTURE WORK

- α/r ratio study
- Patch W_o and FFN layers beyond Q/K/V
- Wider rank sweep
- merge_weights(): fold B@A into W_0
- Downstream tasks beyond SST-2
- Train RoBERTa FT baseline at 10 epochs for direct comparison

REFERENCES

[1] Hu et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685.

[2] Liu et al. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach. arXiv:1907.11692.

[3] Socher et al. (2013). Recursive Deep Models for Semantic Compositionality. EMNLP.

[4] Wolf et al. (2020). HuggingFace Transformers. EMNLP System Demos.

[5] Loshchilov & Hutter (2019). Decoupled Weight Decay (AdamW). ICLR.

CS 4782 Spring 2026, Cornell University.
GPU: Google Colab T4/A100.
Libraries: HuggingFace Transformers & Datasets.